

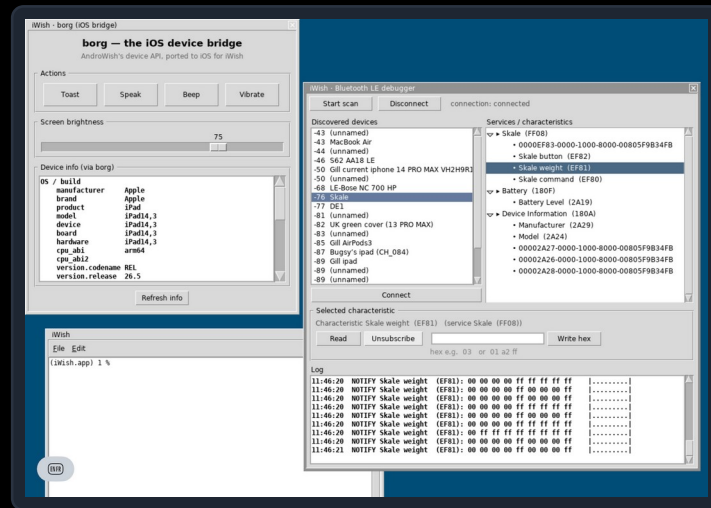
EUROTCL 2026

iWish for iOS

Running real Tcl/Tk applications on iPhone and iPad — from the newest devices back to iOS 9

<https://github.com/johnbuckman/iwish>

John Buckman EuroTcl 2026 · Vienna



MOTIVATION

Tcl/Tk runs everywhere — except in your pocket

- Tk is a first-class GUI toolkit on Linux, macOS and Windows
- Mobile has been the great blank spot for two decades
- AndroWish brought Tk to Android via SDL2 — iOS stayed out of reach
- Apple's rules: no JIT, static linking, code-signing, sandboxing
- Yet the users we care about carry iPhones and iPads



The goal

One *wish* binary that launches your existing Tcl/Tk app on iOS — no rewrite, no web view.

01

From AndroWish to iWish

A single SDL2-backed Tk that already crossed one mobile platform



Standing on the shoulders of undroidwish



AndroWish

Tcl/Tk on Android via SDL2.
Proved that Tk's render loop can
live on a mobile GPU surface
and touch input.



undroidwish

The desktop single-file 'vwish'
built on the same SDL2 core —
the launch pad for new targets,
including arm64 macOS.



iWish

The iOS target: statically linked,
code-signed, App-Store-shaped
— carrying Tk, extensions and
demos in one bundle.

How the stack folds onto iOS

Your Tcl/Tk application · profiles, skins, demos

Tk widgets + canvas · Tcl 8.6 core

SDL2 · window, events, GPU surface

Metal / UIKit · iOS app lifecycle + sandbox

- Statically linked — no dlopen, satisfies Apple's rules
- Tk draws into an SDL2 texture, presented by Metal
- Touches, keyboard and pencil arrive as SDL events
- The whole app is one signed .ipa

Batteries included — always

Never ship a bare wish.

Every iWish bundle carries the extensions and demo apps a Tcl developer expects — so it runs something the moment it launches.

tls

sqlite

tkpath

vfs

tdom

tcllib

BLE

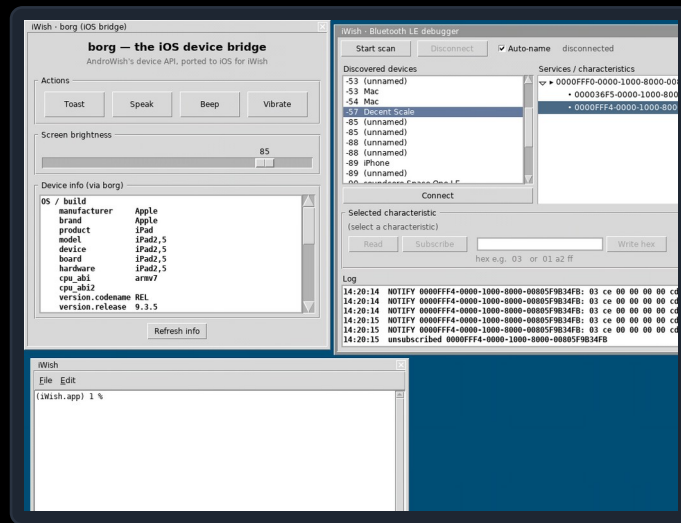
Img

Demos

bundled in every build

All the way back to iOS 9 and armv7

- iPad mini 1 and other 32-bit devices still in daily use
- Built a 32-bit armv7 slice — no modern toolchain assumes it
- BLE on the old stack: solved and shipping
- Self-update needed HTTPS on an ancient TLS — backported
- [Pushed to github.com/johnbuckman/iwish · ios9-armv7](https://github.com/johnbuckman/iwish)

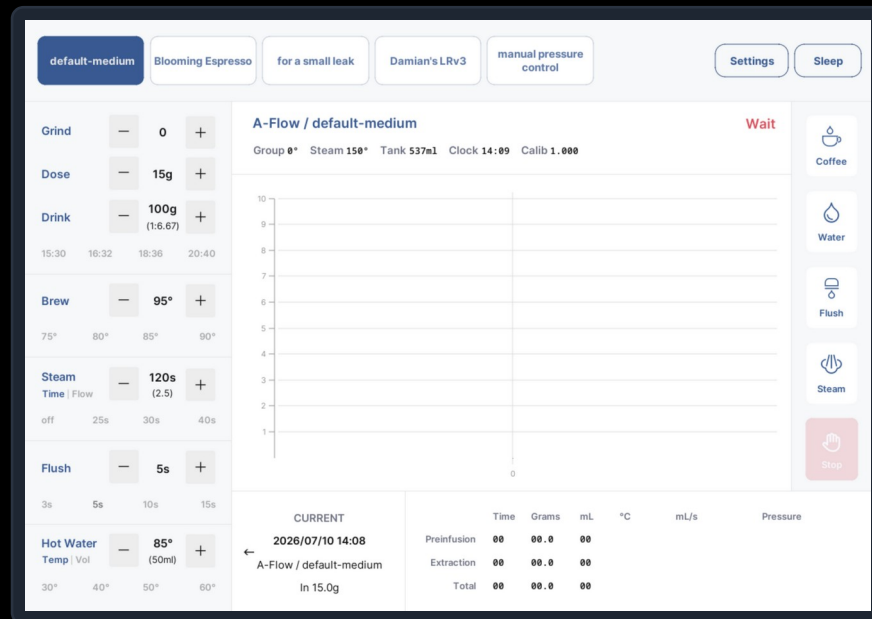


iPad2,5 · armv7 · iOS 9.3.5

CASE STUDY

de1app — an espresso machine in your hand

- Decent Espresso's control app is 100% Tcl/Tk
- Tens of thousands of lines, live BLE to the machine
- iWish runs it unmodified on iPhone and iPad
- Real-time shot graphs via the accelerated canvas
- Shipped to real customers — not a tech demo



Would Apple approve a Tk app?



DE1 app — likely approved

- Full-featured and single-purpose
- Polished, native-looking UI
- Talks to real hardware over BLE
- Self-contained — all scripts baked in



Bare wish — likely refused

- 2.5.2 — an app can't download or run code that changes it
- A 'run any Tcl' launcher or code self-update breaks that
- 4.2 — a bare toolkit is 'minimum functionality': a dev tool

Rule of thumb: ship an app, not an interpreter.

THE 32-BIT CATCH

Jailbreak: the only way onto the mini 1

Apple's signing and current Xcode can't target iOS 9 / armv7 — so the iPad mini 1 had to be jailbroken to run unsigned code.



Advantages

- Installs unsigned IPAs — no account needed
- No 7-day expiry; the app just stays
- Reaches hardware Apple's tools dropped
- Full filesystem access for debugging



Disadvantages

- Only old, exploitable devices qualify
- Disables Apple's security protections
- You can't ask customers to do it
- Fragile — a reboot or update can undo it

BLE, the iOS way

```

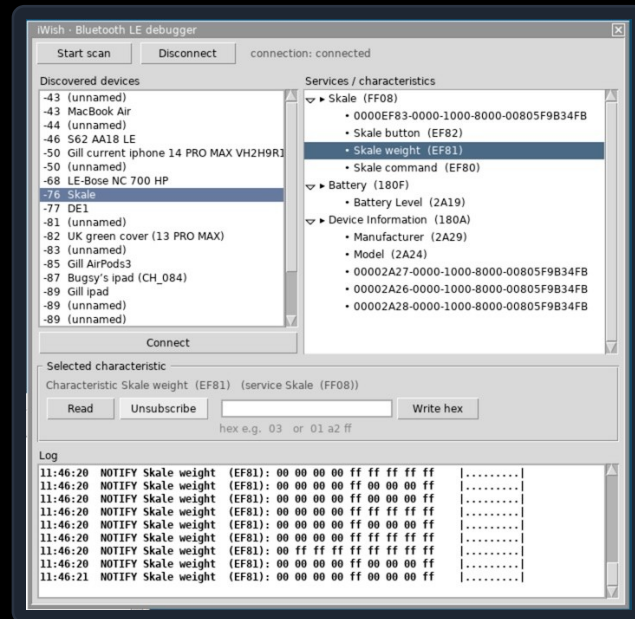
● ● ● connect.tcl

package require ble
ble scan start
ble connect $del_mac
ble subscribe $uuid on_notify

proc on_notify {char data} {
    decode_shot $data
}

# CoreBluetooth under the hood -
# the same ble API as desktop

```



The borg command

AndroWish's borg command — one Tcl call into the native device, now on iOS.

- **Toast, speak, beep, vibrate** — instant user feedback
- Read and set screen brightness
- Device info: model, board, CPU ABI, OS version
- The same borg your AndroWish code already calls
- Native iOS frameworks behind a familiar Tcl API



BY THE NUMBERS

What iWish covers today

iOS 9 → 18

Supported OS range, one
codebase

**armv7 +
arm64**

32- and 64-bit device slices

1 .ipa

App, Tk, extensions and demos
bundled

02

Running & shipping your own apps

Getting a tcl/Tk app onto real iPads — and living with Apple's rules



Seven days free, or a year for \$99



Free Apple ID

7 days

- The signed app stops launching after a week
- Re-sign and re-deploy every 7 days
- Only three sideloaded apps at a time
- Fine for yourself and for development



Paid — \$99 / year

1 year

- Provisioning stays valid a full year
- Install on up to 100 registered devices
- Unlocks TestFlight and the App Store
- The practical choice for real distribution

DISTRIBUTION

Getting your IPA onto other phones



Ad Hoc

Up to 100 devices a year, by UDID. One-year profile, paid account.



TestFlight (beta)

Up to 10,000 testers, but a beta review first; builds expire in 90 days.



Enterprise · \$299/yr

Internal apps for your own staff only — never the public, or Apple revokes it.



App Store

Widest reach, but full review — every guideline applies to you.

INSTALLING IT

Five ways onto your iPad

Route	What you need	Validity
iSideload (USB)	Mac + USB + free Apple ID	7 days free · 1 year paid
Sideloadly	Mac or Windows + Apple ID	7 days free · 1 year paid
SideStore	iPad + one-time pairing	Renews on-device
Build from source	Mac + Xcode toolchain	7 days free · 1 year paid
EU web distribution	An EU device — no computer	Permanent

Full step-by-step guide: github.com/johnbuckman/iwish/blob/main/INSTALL.md

The one-click installer I built

1

Sign in with a free Apple ID

Straight into the menu-bar app

2

Drop in the .ipa

zsign signs it with your Apple ID

3

Signed inside-out

Every nested dylib, not just the wrapper — the bit AltStore skips

4

Plug in over USB

Bundled libimobiledevice installs it — no Xcode, no Python

5

Leave it running

Re-signs iWish every 7 days, automatically

Free · open source · works today — github.com/johnbuckman/iSideload

Toward a cable-free install

1

Registered-device QR — works today

Scan → the signed app installs over the air

2

QR reports the device's ID

Sends its UDID back for ad-hoc registration

3

NaviServer + zsign, fully hosted

Nothing on your Mac — just scan a QR



Over-the-air

The Mac shows a QR of one-tap install links; the iPad pulls the signed app over HTTPS — no cable. Needs Developer Mode on once.

The simple one-time install

1

Download Sideloadly

Grab it free from sideloadly.io

2

Windows only: add Apple's iTunes + iCloud

The versions from apple.com — not the Microsoft Store

3

Connect the iPad over USB

Unlock it and trust the computer

4

Drag in the .ipa and sign in

Sideloadly signs it with your Apple ID

5

Trust the certificate

Settings → General → VPN & Device Management

Free Apple ID → every 7 days · paid → yearly. Its wireless (VPN) mode was finicky for me — USB is the reliable path.

AltStore can't sign for modern iOS

1

Its signer is obsolete

Signs via an Apple ID flow Apple has retired

2

iWish breaks it anyway

~64 native dylibs live outside Frameworks/

3

installld rejects the bundle

Mismatched team signatures → 0xe8008001



What replaced it?

SideStore re-signs on-device — no computer after the first pairing. It should be the successor, but its VPN config defeated me too.

Build it yourself

1

Clone the sources

iWish, AndroWish and SDL2

2

Apply the patches

AndroWish + SDL2, patched for iOS

3

Run the provided build scripts

The whole toolchain is scripted

4

Sign and deploy

7 days free, or a year paid



Scripted end to end

Patch, build and sign with the scripts in the repo — the exact commands live in INSTALL.md.

The DMA cracked sideloading open

- Since iOS 17.4, EU users can install apps from outside the App Store
- Via alternative marketplaces or web distribution
- No 7-day expiry, no 100-device cap — a real download
- But it needs an EU-based developer entity + Apple's alternative terms
- And every app is still Notarized by Apple before it ships

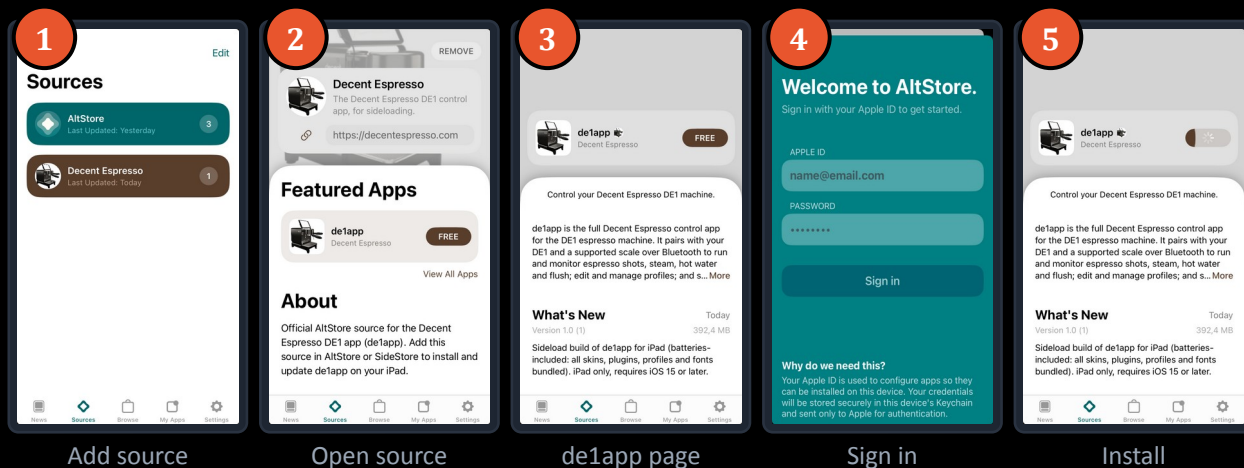


Notarization

An automated security + functionality check — not a content review. But a broken or interpreter-shaped app can still be turned away here.

CAUTION · ALTSTORE

How AltStore used to install it



SideStore — the on-device successor (no computer after pairing) should work, but its VPN setup defeated me too. iSideload is the path that works today.

These screenshots are historical — this flow no longer signs for current iOS. github.com/johnbuckman/iSideload

CALL FOR COLLABORATORS

Wanted: an EU Company (or Org) to help distribute a free Tcl/Tk without limitations

- I'm looking for a partner with an EU-registered company
- ...willing to sign Apple's distribution paperwork
- Then we ship a notarized IPA for open sideloading
- Europeans get iWish + the DE1 app — no 7-day expiry, no device cap
- You're the EU entity of record; I bring the app and the work



Let's talk

Find me at EuroTcl — during the break or after the talk.

Your Tcl/Tk app already runs on iOS



- github.com/johnbuckman/iwish — source and the ios9-armv7 branch
- Bring your wish app; iWish supplies the runtime
- Have an EU company? Let's ship a sideloadable IPA together